LINUX / SYSTEMS

Linux Networking Commands Visual Guide

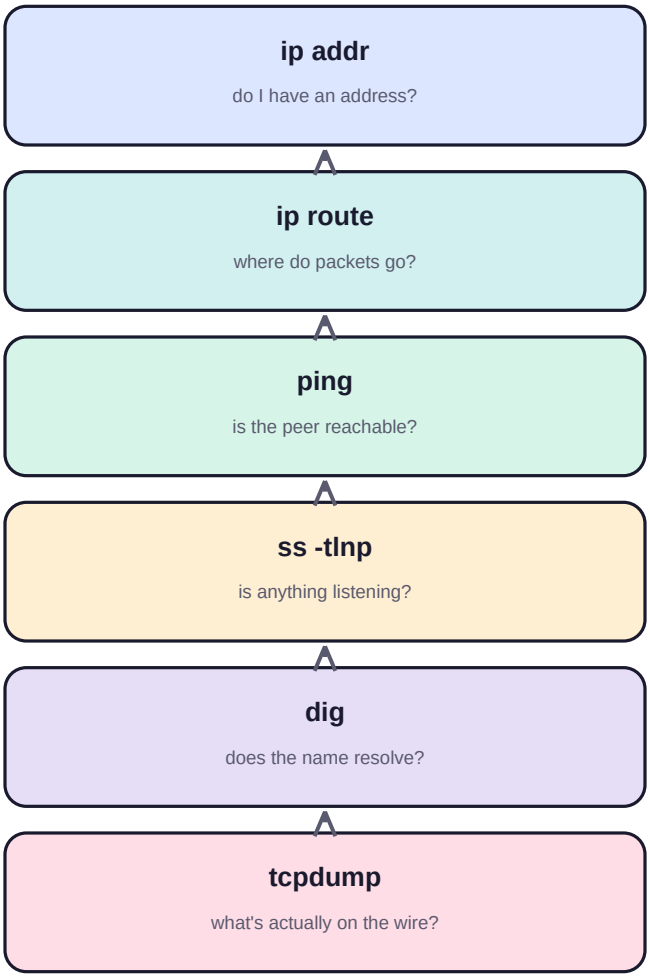
An ip / ss / tcpdump / dig workflow
for diagnosing connectivity, step by step.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Diagnostic Ladder

Always climb bottom-up: local config → route → peer → ports → DNS → packets.



Skipping rungs wastes hours: checking DNS before confirming you have an IP is the classic trap. Each rung eliminates a whole layer.

STEP 2/6

ip: Addresses, Links, Routes

The modern replacement for ifconfig / route / arp.

```
$ ip -brief addr            # addresses at a glance
$ ip route show default    # where outbound traffic goes
$ ip neigh                 # ARP table: IP ↔ MAC
$ ip -s link               # interface counters + errors
$ sudo ip addr add 10.0.0.5/24 dev eth0   # temporary
```

- No address on the interface? Nothing above this rung can work — fix DHCP or static config first.
- Missing default route = 'I can ping LAN but not the internet.' Check ip route.
- RX/TX errors or drops climbing in ip -s link → suspect cable, duplex, or driver.

STEP 3/6

ss: Who's Listening, Who's Connected

Faster, richer netstat. Memorize -tulnp.

```
$ ss -tlnp            # TCP listening sockets + owning processes
$ ss -tup state established   # live connections
$ ss -s               # socket summary counters
```

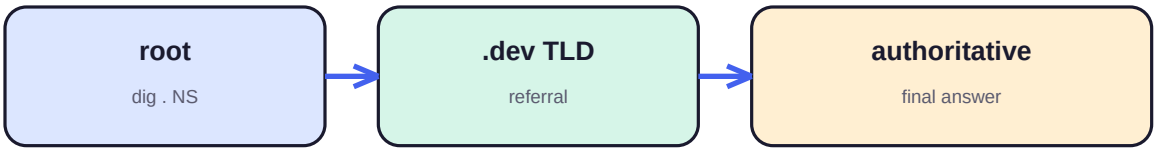
Symptom	ss tells you	Meaning
connection refused	nothing LISTEN on that port	service down / wrong port
hangs, no refuse	SYN-SENT stuck	firewall dropping, or routing blackhole
works locally only	listens on 127.0.0.1	bind address too narrow

Read the flags: t=TCP u=UDP l=listening n=numeric (no DNS lag) p=process. -tlnp is the single most useful networking command on a server.

STEP 4/6

dig: Interrogate DNS Directly

Ask specific servers, follow the delegation chain.




```
$ dig +short example.dev A          # just the answer
$ dig @8.8.8.8 example.dev          # ask a specific resolver
$ dig +trace example.dev            # walk the whole chain yourself
$ dig -x 93.184.216.34             # reverse lookup
```

+trace is the lie detector: it shows every delegation, exposing lame delegations and stale glue your resolver hides.

STEP 5/6

tcpdump: Read the Wire

When tools disagree, capture packets and look.

```
$ sudo tcpdump -i eth0 -n port 443 -c 20
$ sudo tcpdump -i any 'tcp[tcpflags] & tcp-syn != 0'   # SYNs only
$ sudo tcpdump -i eth0 host 10.0.0.8 and port 53 -w dns.pcap
```

- -n: skip DNS resolution (faster, no observer effect). -c: stop after N packets.
- Filter language (BPF): host, port, net, and/or/not — capture less, see more.
- See SYNs with no SYN-ACKs? The far side or a firewall is dropping you. See RSTs? Actively refused.
- Save with -w, analyze later in Wireshark — tcpdump for capture, GUI for forensics.

STEP 6/6

Putting It Together: 'Can't Reach the App'

A 90-second triage script.

```
$ ip -brief addr && ip route show default   # 1. local stack sane?
$ ping -c2 <gateway> && ping -c2 8.8.8.8  # 2. L3 works?
$ ss -tlnp | grep :8080                   # 3. app listening?
$ dig +short app.internal                 # 4. name resolves?
$ curl -sv http://app.internal:8080/      # 5. app answers?
$ sudo tcpdump -i any port 8080 -c 30     # 6. packets prove it
```

Document each rung's result as you go — 'ping OK, ss shows LISTEN on 127.0.0.1 only' is a diagnosis, not a shrug.

STEP ✓/✓

Key Takeaways

Linux Networking Commands

- Diagnose bottom-up: ip addr → ip route → ping → ss → dig → tcpdump. Never skip rungs.
- ss -tlnp replaces netstat — listening sockets plus owning processes in one shot.
- dig +trace walks the delegation chain yourself; don't trust a single resolver's word.
- tcpdump with BPF filters is the ground truth when tools disagree — capture, then analyze.
- Record each rung's verdict; a triage log turns 'it's down' into a diagnosis.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev